

# Data Science

---

## Lecture 09: Association Rule Mining

Prof. Dr. Michael Granitzer

# Module 1

---

## Itemsets and Their Search Space

***Guiding Question:** Ten thousand products give  $2^{10000}$  candidate itemsets. How can this problem possibly be solvable?*

# The Question

Given a set of transactions — shopping baskets, clicked pages, co-occurring symptoms — which **items appear together** more often than chance, and which **imply** which?

## Support

$$P(A \cap B) > \sigma$$

How often do these items occur in the same transaction?

## Confidence

$$P(A \mid B) > \gamma$$

Given  $B$  is in the transaction, how often is  $A$  too?

Both are quantities the students already know: support is a **joint** probability, confidence a **conditional** one, both estimated by counting transactions.

# Formalisation

Let  $I$  be a set of **items** and  $X$  a set of **transactions**, each  $x \subseteq I$ .

- An **itemset** is any  $A \subseteq I$  a  **$k$ -itemset** has  $|A| = k$ .
- Its **support** is  $\sigma(A) = \frac{|\{x \in X : A \subseteq x\}|}{|X|}$ .
- $A$  is **frequent** if  $\sigma(A) \geq \sigma_{\min}$ .

# The Search Space

All itemsets over  $I$  form a **lattice** ordered by inclusion — with  $2^{|I|}$  elements.

A supermarket with 10,000 products has  $2^{10000}$  candidate itemsets. Enumeration is impossible; the entire chapter is about avoiding it.

Support and confidence are joint and conditional probabilities estimated by counting. The difficulty is not the measure but the **size of the search space**.

# Module 2

---

## The A-Priori Algorithm

**Guiding Question:** *What single structural fact about support lets us discard most of an exponential space without looking at the data?*

# The Property That Makes It Possible

**A-priori property (anti-monotonicity).** Every subset of a frequent itemset is frequent — equivalently: if an itemset is **infrequent**, **every superset of it is infrequent too**.

Adding an item can only ever *reduce* the number of transactions containing the set. So an infrequent itemset prunes its entire superset subtree from the search.

# The Algorithm

## A-Priori

$L^1$  = all frequent 1-itemsets.

**for**  $k = 2$   $L^{k-1} \neq \emptyset$   $k++$ : **generate** candidates  $C_k = \{a \cup b \mid a \in L^{k-1}, b \in L^1, b \not\subseteq a\}$   
**prune** any candidate having an infrequent subset **count** the support of the survivors in one pass over the transactions  $L^k = \{c \in C_k \mid \sigma(c) \geq \sigma_{\min}\}$

### Generate

build  $k$ -candidates from frequent  $(k-1)$ -itemsets

### Prune

discard candidates with any infrequent subset — no data access needed

### Count

one database pass per level

# Cost and Improvements

## What it costs

One pass over the transactions **per level  $k$** , plus candidate generation. The bottleneck is I/O and the number of candidates.

## How it is improved

Hashing candidates into buckets, transaction reduction (dropping transactions containing no frequent itemset), partitioning and sampling.

# Mini-Exercise

$\{\text{bread, milk}\}$  has support 0.18, and  $\sigma_{\min} = 0.22$ .

1. Is  $\{\text{bread, milk}\}$  frequent?
2. What can you say about  $\{\text{bread, milk, eggs}\}$ ?
3. How many database passes did answering 2 require?

# Mini-Exercise — Solution

A-Priori turns one structural fact — **infrequent sets have no frequent supersets** — into a level-wise search that prunes most of an exponential space without touching the data.

# Mini-Exercise — Solution

1. No —  $0.18 < 0.22$ .

2. It is **infrequent**, necessarily: its support cannot exceed that of any subset.

3. **None.** The conclusion follows from the property alone. That is exactly where the savings come from.

A-Priori turns one structural fact — **infrequent sets have no frequent supersets** — into a level-wise search that prunes most of an exponential space without touching the data.

# Module 3

---

## From Itemsets to Rules

**Guiding Question:** *A rule holds 80% of the time — and tells you nothing. How is that possible, and what should you measure instead?*

# Generating Rules

An **association rule**  $A \Rightarrow B$  (with  $A \cap B = \emptyset$ ) has

$$\text{support} = \sigma(A \cup B), \quad \text{confidence} = \frac{\sigma(A \cup B)}{\sigma(A)}$$

Every frequent itemset  $F$  is split in all possible ways into  $A$  and  $B = F \setminus A$  rules meeting the confidence threshold are kept.

Support comes from the itemset and is already known. Only confidence must be computed — so rule generation needs **no further database passes**.

# Confidence Is Not Enough

Suppose 80% of all transactions contain milk. A rule  $\text{bread} \Rightarrow \text{milk}$  with **confidence 0.80** has learned nothing: milk appears in 80% of baskets regardless of bread.

$$\text{lift}(A \Rightarrow B) = \frac{P(B \mid A)}{P(B)} = \frac{\sigma(A \cup B)}{\sigma(A)\sigma(B)}$$

## lift > 1

$A$  makes  $B$  **more** likely — a positive association.

## lift = 1

$A$  and  $B$  are **independent** — the rule carries no information.

## lift < 1

$A$  makes  $B$  **less** likely — the rule holds in reverse.

Lift is the independence test from Lecture 02 written as a ratio: it compares the joint probability against the product of the marginals.

# Mini-Exercise

A retailer reports: nappies  $\Rightarrow$  beer, support 0.02, confidence 0.60. Beer appears in 55% of all transactions.

1. Compute the lift.
2. Is the rule actionable?
3. What would make it more convincing?

# Mini-Exercise — Solution

Confidence alone rewards **popular** consequents. **Lift** corrects for the base rate by comparing against independence — the same correction Bayes' theorem makes for priors.

# Mini-Exercise — Solution

**1.**  $\text{lift} = 0.60/0.55 \approx 1.09$  — barely above 1.

**2.** Hardly. A 9% relative increase over the base rate, on 2% of transactions, is a weak effect on a small slice.

**3.** A higher lift, a larger support, and evidence that it **persists** — across stores, seasons and time. And a mechanism: a rule with no explanation is a correlation.

Confidence alone rewards **popular** consequents. **Lift** corrects for the base rate by comparing against independence — the same correction Bayes' theorem makes for priors.